C ASSIGNMENTS

Table of Contents

Bitwise Operators	2
Control Statements:-	5
Patterns:-	8
Arrays	14
STACK	17
Pointers	19
Recursion Functions :-	49
Strings :-	51
FILES	55
C Interview Questions	57

Bitwise Operators

---

0. WAP to check status of a given bit of a Number.

1. Write a program for the following one.

   a) Set a bit b) Clear a bit c) Toggle a bit

2. WAP to find the given number is even or odd using bitwise operators.

3. WAP to find the given number is +ve or -ve using bitwise operators.

4. WAP to swap two numbers using bitwise operators.

5. WAP to find the given number is power of 2 or not.

6. WAP to find the given number is divisble by 8 or not using bitwise operators.

7. Write a program to rotate the bits. Input the no.of rotations, at runtime.

   Ex : binary : 10000000000000000000000000001011

   rotations : suppose 3 times right, then

   result : 01110000000000000000000000000001

   binary : 10000000000000000000000000001011

   rotations : suppose 4 times left, then

   result : 00000000000000000000000010111000

8. Convert the characters Upper to Lower and Lower to Upper using bitwise operators.

9. Write a program to reverse the bits of a given number.

   Note : not just reverse printing.

10. Write a one line code to compare two numbers using bitwise operators.

11. Write a program to print float binay formation using char *ptr.

12. Write a program to swap the adjacent bytes of a given 4-digit hexadecimal number.

    Ex : given number = 0x1234;

    after swap : 0x3412;

13. Write a program to delete no of bits from particular position in a given number.

    Input the no of bits, at runtime.

    Ex: Suppose num = 100;

    It's Binaray is 00000000000000000000000001100100

    delete 2 bits from 4th position

    then result is 00000000000000000000000000011100

14. Write a macro for swapping first and last nibbles in a given integer.

    Ex: Suppose num = 10

    It's Binary is 00000000000000000000000001010

    After swap 10100000000000000000000000000

15. Write a logic to extract P bits from Posion N in an integer M

16. Write a macro to clear a bit at the position N in an integer M.

17. There are 48 bits are stored in an array of character buffer and store them into 2 integer variables.

---------- END ----------

Control Statements:-

---

1) Write a C Program to Find the factorial of a given number. User has to take the i/p at runtime.

2) Write a C Program to Find the sum of digits of a given number.

3) Write a C Program to reverse the digits of a given number.

4) Write a C program to convert a character. If it is Lower, convert it to Upper. and if it is Upper convert it to Lower character.

5) Write a C program to print multiplication tables from 10 to 20.

6) Write a C program to print first 100 prime numbers.

7) Write a C program to print ArmStrong Numbers between 1 to 500.

8) Write a C program to print the binary of a given number (+ve or -ve numbers).

9) Write a C program to reverse the bits of a given number.

10) Write a C program to implement a Calculator using switch.

11) Write a C program to find the complement of a given number and then print it's binary, decimal, octal and Hexa decimal values.

Note : number can be either +ve or -ve. And observe the result satiesfying it's

formulae or not.

Formulae : 1's complement of (x) = - (x+1)

12) Write a C program to find the 2's complement of a given number and then print it's binay, decimal, octal and Hexa decimal values.

Note : number can be either +ve or -ve. And obeseve the result satiefying it's formulae or not.

Formulae : 2's complement of (x) = - (x)

13) Write a C program to find out power of given number without using pow() function.

14) Write a C program to find the complement of a perticular bit. User has to take the position number at runtime.

15) Write a C program to print palindrome numbers between 1 to 1000.

(palindrome numbers means the numbers which are equal to its reverse.

Ex : 11,22,33,44,.....999.)

16) Write a C program to print fibonacci series between 0 to 50.

17) Write a C program to find the given number is power of 2 or not.

18) Write a C program to find out the biggest number of three variables using if-else ladder and terinary operator.

19) Accept a month in digit from the user. Display the month in words. If number is not between 1 and 12 display message “Invalid Month”. (Use ‘switch’)

20) Write a C program to find the given number is Perfect number or not?

Note : Perfect number means sum of it's divisers exept that num is equalent to the same number.

Ex : i/p num = 6.

6 diviesers are = 1,2,3, & 6.

sum = 1+2+3

sum = 6. So here 6 is perfect number.

---------- END ----------

Patterns:-

---

1)

```
        *
       * *
      * * *
     * * * *
    * * * * *
   * * * * * *
    * * * * *
     * * * *
      * * *
       * *
        *
```

2)

```
        *
       ***
      *****
     *******
    *********
   ***********
  *************
   ***********
    *********
     *******
      *****
       ***
        *
```

3)

```
* * * * * *
 * * * * *
  * * * *
   * * *
    * *
     *
    * *
   * * *
  * * * *
 * * * * *
* * * * * *
```

4)

```
***********
 *********
  *******
   *****
    ***
     *
    ***
   *****
  *******
 *********
***********
```

5)

1

2*2

3*3*3

4*4*4*4

5*5*5*5*5

4*4*4*4

3*3*3

2*2

1

6)

1

1 0

1 0 1

1 0 1 0

1 0 1 0 1

1 0 1 0 1 0

7) 1

2 4

1 3 5

2 4 6 8

1 3 5 7 9

2 4 6 8 10 12

8) 1

2 6

3 7 10

4 8 11 13

5 9 12 14 15

9) 1

1 2 3

1 2 3 4 5

1 2 3 4 5 6 7

1 2 3 4 5 6 7 8 9

10) 1

1 2 3

1 2 3 4 5

1 2 3 4 5 6 7

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7

1 2 3 4 5

1 2 3

1

11)

```
1
2 4
3 6 9
4 8 12 16
5 10 15 20 25
6 12 18 24 30 36
```

12)

```
1                     1
1 2                 2 1
1 2 3             3 2 1
1 2 3 4       4 3 2 1
1 2 3 4 5 5 4 3 2 1
```

13)

```
1                   1
1 2               2 1
1 2 3           3 2 1
1 2 3 4     4 3 2 1
1 2 3 4 5 4 3 2 1
```

14)

```
A B C D E F G F E D C B A
A B C D E F   F E D C B A
A B C D E       E D C B A
A B C D           D C B A
A B C               C B A
A B                   B A
A                       A
```

15)

```
        1
      1  1
    1  2  1
  1  3  3  1
1  4  6  4  1
1  5  10 10 5 1
```

---------- END ----------

Arrays

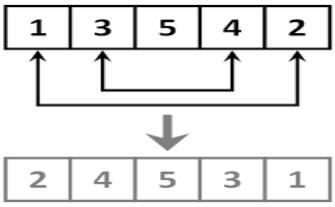
1) Write a C program to input 10 numbers through the keyboard into an array and display the results of addition of even numbers and product of odd numbers.

2) Write a C program to input 10 numbers through the keyboard into an array and find the biggest and smallest number in an Unsorted array without using any Sorting Technique.

3) Write a C program to input 10 numbers through the keyboard and find the number of prime numbers count, store them into a seperate array and display it.

4) Write a C program to findout second largest and second smallest elements of an unsorted array without using any Sorting Technique.

5) Write a C program to reverse the elements of a given array.



6) Write a C program to delete an element at desired position from an array.

1	14
2	50
3	73
4	9
5	24
6	3
7	92
8	-3

Original Array

1	14
2	50
3	73
4	9
5	
6	3
7	92
8	-3

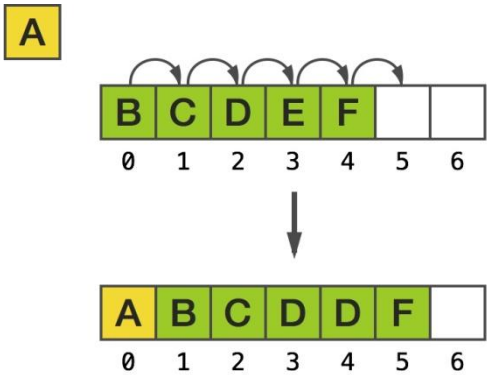
5th Element deleted – leaving an empty location

1	14
2	50
3	73
4	9
5	3
6	92
7	-3
8	

Array after Deletion

7) Write a C program to insert an element at desired position in an array.

For Example if 'A' is to be stored at '0' position then,



8) Write a C program which deletes the duplicate elements of an array.

Original: [A, C, B, D, A, B, E, D, B, C]
Remove duplicate result: D, E, A, B, C,

javazs.com

9) Write a C program to find the duplicate elements of a given array and find the count of duplicated elements.

Ex: if int a[] = {0,3,1,0,5,1,2,0,4,5}

output : -

The duplicate elements are existed in an array

0 -- 3 times

1 -- 2 times

5 -- 2 times

10) Write a program to print the non repeted numbers of a given array.

Ex : if int a[] = {0,3,1,0,5,1,2,0,4,5}

Output : 3, 2, 4

11) Write a program to copy the elements of one array into another array without duplicate items as a first slot, and store duplicate elements as a second slot.

Ex: source array {10,2,4,5,2,1,3,4,6,5,8,9,2}

destination arrays {10,2,4,5,1,3,6,8,9} , { 2,2,4,5}

first slot second slot

Take two different arrays for first and second slots.

12) Write a C program to evaluate the following series. The series contains sum of square of numbers from 1 to 'n'. Strore result of each term in an array. Calculate value of ' S ' using array.

S = 1^2 + 2^2 + 3^2 + 4^2 + ------ n^2

= [1, 4, 9, 16, -------- n^2]

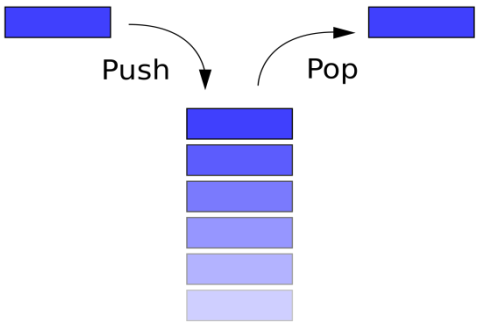
Suppose n = 4,

then S = 1^2+2^2+3^2+4^2;

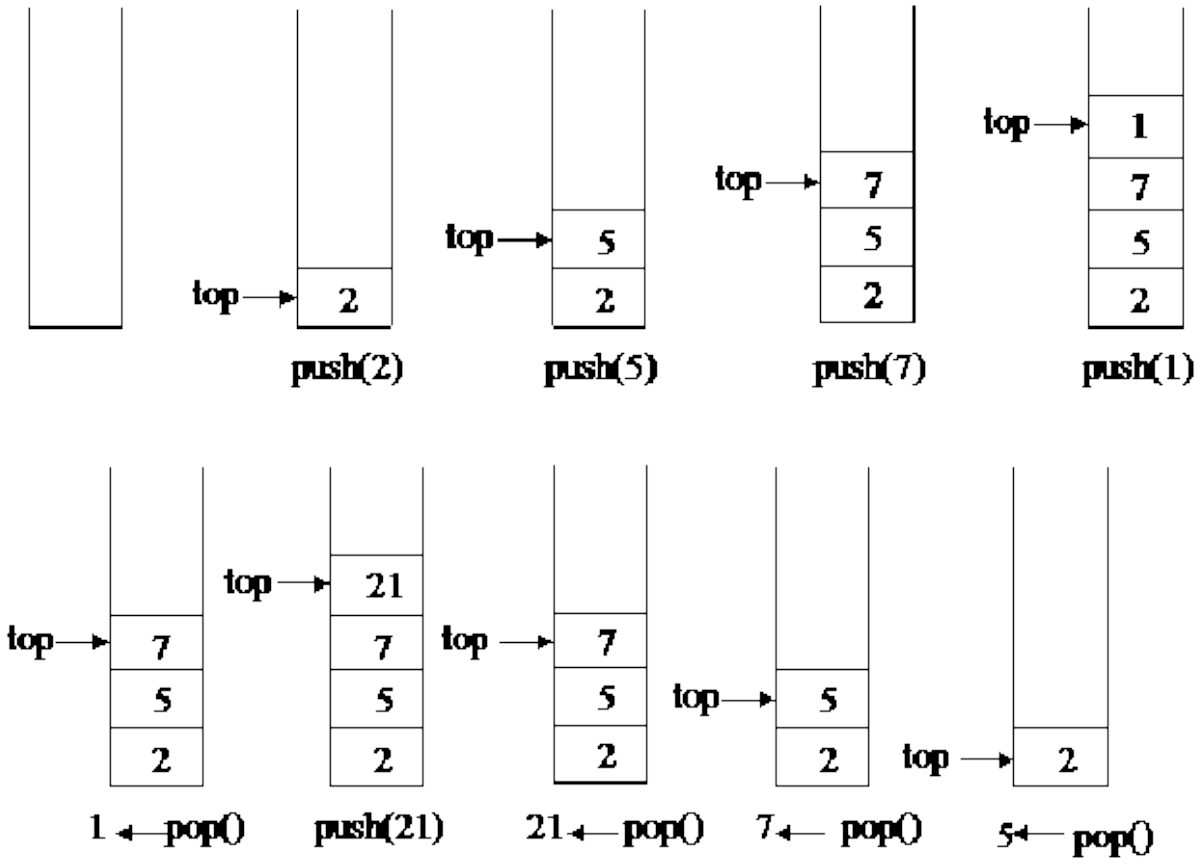
S = 1+4+9+16;

S = 30.

13) Write a C program to implement the stack using arrays.



STACK


top
2
push(2)
top
5
2
push(5)
top
7
5
2
push(7)
top
1
7
5
2
push(1)
top
7
5
2
1 ← pop()
top
21
7
5
2
push(21)
top
7
5
2
21 ← pop()
top
5
2
7 ← pop()
top
2
5 ← pop()

---------- END ----------

Pointers

Find out the output for the following programs.

1)
```
#include <stdio.h >
int main()
{
    int *p = 10;
    printf(“ %u\n”, (unsigned int)p);
    printf(“%d\n”,*p);
}
```

2)
```
#include <stdio.h>
int main()
{
    int *ptr, a = 10;
    ptr = &a;
    *ptr += 1;
    printf("%d,%d/n", *ptr, a);

}
```

3)
```
#include<stdio.h>
int main()
{
    int x = -300;
    unsigned char *p;
    p = &x;
    printf(“%d\n”,*p++);
```

```
        printf("%d\n",*p);
    }
```

4)
```
#include<stdio.h>
int main()
{
    int x = 256;
    char *p = &x;
    *++p = 2;
    printf("%d",x);
}
```

5)
```
#include<stdio.h>
int main()
{
    int x = 300;
    if(*(char *)&x == 44)
    printf("Little Endian\n");
    else
    printf("Big Endian\n");
}
```

6)
```
#include <stdio.h>
void main()
{
    int x = 0;
    int *ptr = &5;
    printf("%p\n", ptr);
}
```

7)
```
#include<stdio.h>
int main()
{
    int const *p = 5;
    int q;
    p = &q;
    printf("%d",++(*p));
}
```

8)
```
#include<stdio.h>
int main()
{
    int x = 10;
    int const * const p;
    p = &x;
    printf("%d\n", *p);
}
```

9)
```
#include <stdio.h>
int x = 0;
void main()
{
    int *const ptr = &x;
    printf("%p\n", ptr);
    ptr++;
    printf("%p\n ", ptr);
}
```

10)
```
#include <stdio.h>
int main()
 {
    const int ary[4] = {1, 2, 3, 4};
    int *p;
    p = ary + 3;
    *p = 5;
    printf("%d\n", ary[3]);
}
```

11)
```
#include <stdio.h>
int main()
{
    int ary[4] = {1, 2, 3, 4};
    int *p = ary + 3;
    printf("%d\n", p[-2]);    }
```

12)
```
#include <stdio.h>
void main()
{
    char *s= "hello";
    char *p = s + 2;
    printf("%c\t%c", *p, s[1]);
}
```

13)
```
#include <stdio.h>
int main()
{
```

```
    void *p;
    int a[4] = {1, 2, 3, 4};
    p = &a[3];
    int *ptr = &a[2];
    int n = (int*)p - ptr;
    printf("%d\n", n);
  }
```

14)
```
#include<stdio.h>
int main()
{
    int a[ ] = {10,20,30,40,50},i;
    char *p = a;

    for(i=0;i<5;i++)
   printf("%d   ",*p++);
}
```

15)
```
#include<stdio.h>
int main()
{
    int a[]={10,20,30,40,50};
    char *p;
    p=(char *)a;
    printf("%d\n",*((int *)p+4));
}
```

16)
```
#include <stdio.h>
int main()
{
    double *ptr = (double *)100;
    ptr = ptr + 2;
    printf("%u\n", ptr);
}
```

17)
```
#include <stdio.h>
int main()
{
    int i = 10;
    void *p = &i;
    printf("%d\n", (int *)*p);
// printf("%d\n", *(int*)p);
    return 0;
}
```

18)
```
#include <stdio.h>
int main()
{
    int a[4] = {1, 2, 3, 4};
    void *p = &a[1];
    void *ptr = &a[2];
    int n = 1;
    n = ptr - p;
    printf("%d\n", n);
```

```
}
```

19)
```
#include <stdio.h>
int main()
{
    int *p = (int *)2;
    int *q = (int *)3;
    printf("%d", p + q);
}
```

20) Which of the following operand can be applied to pointers p and q?

(Assuming initialization as int *a = (int *)2; int *b = (int *)3;)

a) a + b

b) a – b

c) a * b

d) a / b

Ans: b)

21) Which of following logical operation can be applied to pointers?

(Assuming initialization int *a = 2; int *b = 3;)

a) a | b

b) a ^ b

c) a & b

d) None of the mentioned

Ans: d)

22)
```
#include <stdio.h>
void main()
{
    char *s = "hello";
    char *n = "cjn";
    char *p = s + n;
    printf("%c\t%c", *p, s[1]);
}
```

23)
```
#include <stdio.h>
void m(int *p)
{
    int i = 0;
    for(i = 0;i < 5; i++)
    printf("%d\t", p[i]);
}
void main()
{
    int a[5] = {6, 5, 3};
    m(&a);
}
```

24)
```
#include <stdio.h>
void foo(int*);
int main()
{
    int i = 10,j=20,*p = &i;
    foo(p++);
```

```
        foo(p);
    }
    void foo(int *p)
    {
        printf("%d\n", *p);
    }
```

25)
```
#include <stdio.h>
    int main()
    {
        int i = 97, *p = &i;
        foo(&i);
        printf("%d ", *p);
    }
    void foo(int *p)
    {
        int j = 2;
        p = &j;
        printf("%d ", *p);
    }
```

26)
```
#include<stdio.h>
    int main()
    {
        const int ary[4] = {1,2,3,4};
        int *p = ary+3;
        *p = 5;
        ary[3] = 6;
```

```
        printf("%d",ary[3]);
    }
```

27)
```
#include<stdio.h>
 int main()
{
    char *p = "Hai friends",  *p1 = p;
    while(*p!='\0');
    ++*p++;
    printf("%s  %s\n",p,p1);
}
```

28)
```
#include<stdio.h>
 int main()
 {
    char *x = "VECTOR";
    printf("%s\n",x+3);
    printf("%d\n"+1,123456);
 }
```

29)
```
#include<stdio.h>
 int main()
 {
    char a[ ] = "abcdefgh";
    int *ptr = a;
    printf("%x  %x\n",ptr[0],ptr[1]);
```

```
}
```

30)
```
#include<stdio.h>
#include<string.h>
int main()
{
    char *str = "hello, world\n";
    char *strc = "good morning\n";
    strcpy(strc, str);
    printf("%s\n", strc);
    return 0;
}
```

31)
```
#include <stdio.h>
int main()
{
    char *str = "hello world";
    char strc[50] = "good morning india\n";
    strcpy(strc, str);
    printf("%s\n", strc);
    return 0;
}
```

32)
```
#include <stdio.h>
int main()
{
    char *str = "hello, world\n";
    str[5] = '.';
```

```
    printf("%s\n", str);
    return 0;
}
```

33)
```
#include <stdio.h>
int main()
{
    char str[] = "hello, world";
    str[5] = '.';
    printf("%s\n", str);
    return 0;
}
```

34)
```
#include <stdio.h>
int main()
{
    char *str = "hello world";
    char strary[] = "hello world";
    printf("%d %d\n", sizeof(str), sizeof(strary));
    return 0;
}
```

35)
```
#include <stdio.h>
int main()
{
    char *str = "hello world";
    char strary[] = "hello world";
    printf("%d %d\n", strlen(str), strlen(strary));
```

```
    return 0;
}
```

36)
```
#include<stdio.h>
int main()
{
    int a = 5,b = 4,c = 9;
    *(a>b ? &a : &b) = (a+b)>c;
    printf("%d  %d\n",a,b);
}
```

37) Find the sizeof any datatype with out using sizeof operator. (Hint : Use pointers)

38)
```
#include<stdio.h>
int main()
{
    int i;
    double a = 5.2;
    char *ptr;
    ptr = (char *)&a;
    for(i=0;i<=7;i++)
    printf("%d\n",*ptr++);
    return 0;
}
```

39) Correct the following program.

```
#include<stdio.h>
int main()
```

```
{
void *p;
        int  **ptr;
        int a = 129;
p = &a;
        ptr = &p;
printf(“ p = %d   p = %u  &p = %u\n”, *p, p, &p);
}
```

40)
```
#include<stdio.h>
main()
{
    char a[20];
    char *p,*q;
    p=&a[0];
    q=&a[10];
    printf("%d %d\n",q-p,&q-&p);
}
```

41)
```
#include<stdio.h>
main()
{
    int a=0x12345678;
    void *ptr;
    ptr=&a;
    printf("0x%x\n",*(int *)&*&*(char*)ptr);
}
```

```
42) #include<stdio.h>
    main()
    {
        int a[5]={1,2,3,4,5};
        int *ptr=(int *)(&a+1);
        printf("%d %d\n",*(a+1),*(ptr-1));
        printf("%d %d\n",*(a+1),*(ptr));
    }
```

```
43) #include <stdio.h>
      void main()
      {
        char *s= "hello";
        char *p = s;
        printf("%c\t%c", 1[p], s[1]);
      }
```

```
44) #include<stdio.h>
     main()
     {
        char a[]="abcde";
        char *p=a;
        p++;
        p++;
        p[2]='z';
        printf("%s",p);
     }
```

45)
```
#include<stdio.h>
main()
{
char a[]=”ABCDEFGHIJKLMNOPQRSTUVWXYZ”;
        int i,*p = a;
        for(i=0;i<5;i++)
        printf(“%d\t”,*p++);
}
```

46)
```
#include<stdio.h>
main()
{
char *ptr = “abcdef”;
ptr1 = ptr1+(strlen(ptr1)-1);
printf(“%c”, --*ptr1--);
printf(“%c”,--*--ptr1);
printf(“%c”,--*(ptr1--));
printf(“%c”,--*(--ptr1));
printf(“%c”,*ptr1);
}
```

47)
```
#include<stdio.h>
int main()
{
char *str1 = “Hello”;
char *str2 = “Hai”;
char *str3;
str3 = strcat(str1,str2);
printf(“%s  %s\n”,str3,str1);
```

```
return 0;
}
```

48)
```
#include<stdio.h>
int main()
{
char a[]=”Hello”;
char *p=”Hai”;
a=”Hai”;
p=”Hello”;
printf(“%s  %s\n”,a,p);
return 0;
}
```

49)
```
#include<stdio.h>
int main()
{
int i,n;
char *x=”Alice”;
n=strlen(x);
*x=x[n];
for(i=0;i<=n;i++)
{
        printf(“%s”,x);
        x++;
}
printf(“%s\n”,x);
```

```
    return 0;
    }
```

50)
```
#include<stdio.h>
char *str=”char *str=%c%s%c;main(){printf(str,34,str,34);}”;
int main()
{
printf(str,34,str,34);
return 0;
}
```

51)
```
#include <stdio.h>
void f(char *k)
{
    k++;
    k[2] = 'm';
    printf("%c\n", *k);
}
void main()
{
    char s[] = "hello";
    f(s);
    printf("%s\n",s);
}
```

52)
```
#include<stdio.h>
void t1(char *q);
main()
```

```
{
    char *p;
    p = “abcder”;
    t1(p);
}
void t1(char *q)
{
    if(*q!='r')
    {
            putchar(*q);
            t1(q++);
    }
}
```

53)
```
#include<stdio.h>
int main(){
int i;
float a=5.2;
char *ptr;
ptr=(char *)&a;
for(i=0;i<=3;i++)
printf("%d ",*ptr++);
return 0;
}
```

54)
```
#include <stdio.h>
void foo( int[] );
int main()
{
```

```
    int ary[4] = {1, 2, 3, 4};
    foo(ary);
    printf("%d ", ary[0]);
}
void foo(int p[4])
{
    int i = 10;
    p = &i;
    printf("%d ", p[0]);
}
```

55)
```
#include <stdio.h>
void main()
{
    int k = 5;
    int *p = &k;
    int **m  = &p;
     **m = 10;
    printf("%d%d%d\n", k, *p, **m);
}
```

56)
```
#include <stdio.h>
int main()
{
    int a = 1, b = 2, c = 3;
    int *ptr1 = &a, *ptr2 = &b, *ptr3 = &c;
    int **sptr = &ptr1;
        printf("%d  ",**sptr);
```

```
    *sptr = ptr2;
        printf(("%d  ",**sptr);
}
```

57)
```
#include <stdio.h>
void main()
{
    int a[3] = {1, 2, 3};
    int *p = a;
    int *r = &p;
    printf("%d\n", (**r));
}
```

58)
```
#include <stdio.h>
int main()
{
    int i = 97, *p = &i;
    foo(&p);
    printf("%d ", *p);
    return 0;
}
void foo(int **p)
{
    int j = 2;
    *p = &j;
    printf("%d ", **p);
}
```

59)
```
#include <stdio.h>
void foo(int *const *p);
int main()
{
    int i = 11;
    int *p = &i;
    foo(&p);
    printf("%d ", *p);
}
void foo(int *const *p)
{
    int j = 10;
    *p = &j;
    printf("%d ", **p);
}
```

60)
```
#include <stdio.h>
void foo(int **const p);
int main()
{
    int i = 10;
    int *p = &i;
    foo(&p);
    printf("%d ", *p);
}
void foo(int **const p)
{
    int j = 11;
```

```
    *p = &j;
    printf("%d ", **p);
}
```

61)
```
#include <stdio.h>
int *f();
int main()
{
    int *p = f();
    printf("%d\n", *p);
}
int *f()
{
    int *j = (int*)malloc(sizeof(int));
    *j = 10;
    return j;
}
```

62)
```
#include <stdio.h>
void main()
{
    char *a[10] = {"hi", "hello", "how"};
    int i = 0;
    for (i = 0;i < 10; i++)
    printf("%s  ", *(a[i]));
}
```

63)
```
#include <stdio.h>
```

```
void main()
{
    char *a[10] = {"hi", "hello", "how"};
    int i = 0, j = 0;
    a[0] = "hey";
    for (i = 0;i < 10; i++)
    printf("%s  ", a[i]);
}
```

64)
```
#include <stdio.h>
void main()
{
    char *a[10] = {"hi", "hello", "how"};
    printf("%d\n", sizeof(a));
}
```

65)
```
#include <stdio.h>
void main()
{
    char *a[10] = {"hi", "hello", "how"};
    printf("%d\n", sizeof(a[1]));
}
```

66)
```
#include <stdio.h>
int main()
{
    char a[2][6] = {"hello", "hi"};
```

```
    printf("%s  ", *a + 1);
    return 0;
}
```

67)
```
#include <stdio.h>
int main()
{
    char *a[2] = {"hello", "hi"};
    printf("%s\n", *(a + 1));
    return 0;
}
```

68)
```
#include <stdio.h>
int main(int argc, char *argv[])
{
    while (argc--)
    printf("%s\n", argv[argc]);
    return 0;
}
```

69)
```
#include <stdio.h>
int main(int argc, char *argv[])
{
    while (*argv++ != NULL)
    printf("%s\n", *argv);
    return 0;
}
```

70)
```
#include <stdio.h>
int main(int argc, char *argv[])
{
    while (*argv  !=  NULL)
    printf("%s\n", *(argv++));

   return 0;
}
```

71)
```
#include<stdio.h>
int main(int sizeofargv, char *argv[])
{
while(sizeofargv)
printf("%s  ",argv[--sizeofargv]);
return 0;
} if i/p is   sample  friday tuesday sunday
```

72)
```
#include<stdio.h>
int main()
{
char *str[]={"Progs","Do","Not","Die","They","Croak!"};
printf("%d  %d",sizeof(str),strlen(str[0]));
return 0;
}
```

73)
```
#include<stdio.h>
int main()
{
static char *s[]={"black","white","pink","violet"};
```

```
char **ptr[]={s+3,s+2,s+1,s},***p;
p = ptr;
printf("%s\n",**p+1);
return 0;
}
```

74)
```
#include<stdio.h>
main()
{
char *m[]={"jan","feb","mar"};
char d[][10] = {"sun","mon","tue"};
printf("%s\t",m[1]);
printf("%s\t",d[1]);
}
```

75)
```
#include<stdio.h>
void fun(char **);
int main()
{
char *argv[]={"ab","cd","ef","gh"};
fun(argv);
return 0;
}
void fun(char **p)
{
char *t;
t=(p+=sizeof(int))[-1];
printf("%s\n",t);
```

```
    }
```

76)
```
#include <stdio.h>
void first()
{
    printf("first");
}
void second()
{
    first();
}
void third()
{
    second();
}
void main()
{
    void (*ptr)();
    ptr = third;
    ptr();
}
```

77)
```
#include <stdio.h>
int add(int a, int b)
{
    return a + b;
}
int main()
```

```
{
    int (*fn_ptr)(int, int);
    fn_ptr = add;
    printf("The sum of two numbers is: %d\n", (int)fn_ptr(2, 3));
}
```

78)
```
#include <stdio.h>
int mul(int a, int b, int c)
{
    return a * b * c;
}
void main()
{
    int (*function_pointer)(int, int, int);
    function_pointer  =  mul;
    printf("The product of three numbers is:%d",
    function_pointer(2, 3, 4));
}
```

79)
```
#include<stdio.h>
int fun(int (*)());
int main()
{
fun(main);
printf("Hi\n");
return 0;
}
int fun(int (*p)())
```

```
{
printf("Hello\n");
return 0;
}
```

80)
```
#include<stdio.h>
int main()
{
char *p = "Hello World";
printf(p);
}
```

---------- END ----------

Recursion Functions :-

---

1) Write a recursive function to find the factorial of a given number..

2) Write a recursive function to print the 'n' fibonacci series numbers.

3) Write a recursive function to find the sum of digits of a given number.

4) Write a recursive function to revese the given number.

5) Write a recursive function to that displays all the proper divisors of a given number exept that and returns their sum.

   Ex: 1,3,5,9,15 & 45 are the proper divisors of 45.

   sum = 1+3+5+9+15

   = 33

6) Write a recursive function that displays a positive integer in words. For ex: if the integer is 3412 then it is displayed as three four one two.

7) Write a recursive function to print first 100 prime numbers.

8) Write a recursive function to print the palindrome numbers in a given numbers.

9) A number is perfect if the sum of all its positive proper divisors is equal to the number. For example 28 is a perfect number since 28 = 1+2+4+14. Write a recursive function that finds whether a number is perfect or not.

10) Write a recursive function to find the largest element in a given Unsorted array.

11) Write a recursive function to reverse the bits of a given number.

12) Write a recursive function to revese the elements of a given array.

13) Write a recursive function to revese the string. (Note : not just reverse printing charecter by charecter)

---------- END ----------

Strings :-

---

1) Write a program to find string length using pointer.

2) Write a one line code to copy the string into another buffer.

3) Write a program to find the no.of times the character is found in a given string.

4) Write a program to find vowels in a given string.

5) Write a program to compare two strings without using strcmp function.

6) Write a program to reverse the string using loops & recursion.

7) Write a program to check the given strings are palindrom or not.

Note: Palindrome words are words which read and spell the same way both backwards and forwards. Some examples,

madam, level , radar, stats and etc.

8) Write a program to find the no.of words are presented in a given string line.

Note: User has to input the string line at runtime.

9) Write a program to delete a desired character in a given string.

Ex: Input : embedded

character: 'd'

Output : embee

10) Write a program to remove the conjucutive spaces in a given string line.

Ex: Input: Vector India Pvt Ltd

Output: Vector India Pvt Ltd;

11) Write a program to delete the duplicate characters in a given string.

Ex: Input : **vecteeovvorr**

Output : **vector**

12) Write a program to print the count of duplicate characters in a given string.

Ex: Input : **"hrithik roshan"**

Output: Letter ------- Count

h ------- 3

r ------- 2

i ------- 2

13) Write a program to find count of Lower characters, Upper characters, Special characters and digits occured in a given string.

14) Write a program to convert the characters Upper to Lower and Lower to Upper in a given string.

15) Write a program to sort a given string in ascending order.

16) Write a program to accept two strings from user into two character array and copy one by one character into another destination array.

Ex: First String : "abcdefg"

Second String : "1234"

then Destination String is "a1b2c3d4efg"

17) Write a program to find the no.of times substring is found in a given string.

18) Write a program to reverse the words in a given string line.

Ex : "I am a good boy"

"I ma a doog yob"

19) Write a program to replace the words in reverse order in a given string line.

Ex: Input : **"world changed your thoughts"**

Output : **"thoughts your changed world"**

20)Write a program to declare an array of string pointers. Use function "input" to input the strings from stdin into array of string pointers, sort them in assending order according to string length.

Note: Use DMA.

21) Write a program to read two strings through the keyboard like the following example and replace any word of the second string with the first string.

Ex: Input:- Fist String : "Tomorrow"

Second String : "Today is Sunday"

Replace word : "Today".

Output:- "Tomorrow is Sunday"

22) Write a program to check given strings are anagram or not.

Note : Both strings are anagram, if both contains same elements, same no.of times in any order.

(Can have extra special characters & digits also)

Ex: "Osama bin laden" , "Old man in a base" both are anagrams.

"study" , "**dus%@ty123**" both are anagrams.

Here after removing special characters and digits ----> **" dusty "**.

23) Write a program for Decryption of a given Encrypted string line with respect to given

key number.

The letter at position of the key and a multiple of key is interchanged with the next letter. Spaces and special characters are to be ignored. If the letter is the last one in the array, then no interchange is required.

Design a function called Decryptor to receive the Encrypted data from the main function and decrypt the data.

4 8 12 16 20 24 28 32 36

Ex: Input String : **"Expcet Porblmes adn eat thef mor rbeafkast".**

If Key : 4 then, 4 multiples ---> 4,8,12,16,20,24,28,32,36 these letters has to interchange with next Characters.

Output String : **"Expect Problems and eat them for breakfast".**

Try this and Know it.

Input String : "Our rgeaetst ewakenss iles ni giivng pu. thm eosc teratin awy ts ouceed si alwyas tt ory ujst noe mroe tmie".

Key : 4

Output : ????????

FILES

---

1) Write a Program to Convert Lower to Upper and Upper to Lower Cases of a given file.

2) Write a Program to remove a Specific line from the given text file.

3) Write a Program to replace a Specified line in a given file.

4) Write a Program to Capitalize First Letter of every Word in a file.

5) Write a Program to meges the lines from two files and store the result into another file.

6) Write a Program to replace the word with the reverse of that word in a given file.

7) Write a Program to copy the one file into multiple destination files which are provided during the loadtime.

8) Write a Program to replace the perticular word with another word (can be any length) in a given file.

9) Write a Program to reverse the Contents of a given file.

10) Write a Program to implement **sort** command.

    (Check the command: sort filename.c)

11) Write a Program to implement **grep** command.

    (Check the command: grep searching_word filename.c)

12) Write a Program to implement **wc** command.

(Check the command: wc filename.c)

---------- END ----------

C Interview Questions

1) What are the compilation stages and it's commands.

2) What are the types of errors are occured in C?

3) What is code-optimization? What are advantages of code-optimization?

4) Write a program to set a bit, clear a bit , and complement a bits.

5) Write a one line code to compare the two integer numbers using bitwise operator.

6) Write a program that converts Lower to Upper and Upper to Lower characters using bitwise.

7) What is typecasting? What are the types of typecasting? Explain with example.

8) What is the format specifier to use for printing address? Ans : "%p"

9) What are the ristrictions of switch cases? (What you should not use for switch cases).

11) Write a program to satisfy the if and else conditions. Ans : Use goto

12) Write a program to swap two integers using bitwise operators.

13) What are minimum no.of iterations for while loop? Ans : 0

14) What is volatile keyword?

19) What are the differences between Arrays & Pointers?

20) What is the sizeof char *.

21) What is the values for the following.

a) sizeof 123456 b) sizeof "123456"

22) What is Array of pointer? Give one example.

23) What is Pointer to an array? Give one example.

24) What is the difference between Array of pointer and Pointer to Array?

25) What is Function? What are the advantages?

26) What is the difference between Call by Value & Call by reference?

27) What is Function Pointer? What is the advantage of Function Pointer?

28) What is command line arguments? What is the use of that?

29) By default Command line Arguments are treated as ????

Ans : Strings

30) Storage Classes :

a) auto b) static c) register d) extern

Explain the 4 specifications 1) Scope 2) lifetime 3) Default Value 4) Storage.

31) What is default storage class for local variable and global variables?

32) What is memory leak?

33) Can we declare register variable as gloabal?

34) Can we access the addresses of registers?

35) What is static function?

36) What is the default storage class for any function.

37) What is the recursion function? What are the advantages and disadvantages?

38) What are the difference between malloc & calloc?

39) What is the return type of malloc function.

40) What is the use of realloc function.

41) Write a program to allocate memory dynamically without using malloc and calloc functions.

42) Write a program to free the dynamic memory allocation without using free() function.

43) What is memory leak?

44) What is dangling pointer? How to avoid the dangling pointer?

45) What is structure? What are differences between Arrays and Structures?

46) What is structure padding? How to avoid structure padding?

47) Write a program to find the sizeof structure without using sizeof operator?

48) When the memory is created to the structure?

49) What are the return values of strcmp function?

50) Write a one line code to copy string into destination.

51) What are types of qualifiers? Ans : i) const ii) size

52) What is self referential structure pointer?

53) What is enum? What are the advantages?

54) What is typedef? What are the advantages?

55) What is FILE in C? What is the size of that?

56) What is union? What are the advantages?

57) What are differences between Structure and Union?

58) What are the differences between macros and fucntions?

59) What is the difference between #define and typedef?

60) What are the default file pointers of a program?

61) What is the difference between strcpy and memcpy?

62) What does Header file and Libraries contains?

Files(chapter 11,12)_85

1) In _____ access, the larger the block size, the fewer I/O operations required and the more contiguous the disk accesses. Compare loading a single 16K block with loading 32 512-byte blocks.

1) Sequential access
2) Random access
3) Both
4) None

Answer: 1

Level:1

2) The choice of block size affects the file system performance? T/F

Answer: True

Level:1

3) In_____ access, the larger the block size, the more unrelated data loaded. Spatial locality of access can improve the situation.

1) Random access
2) sequential access
3) both
4) none

Answer: 1

Level:1

4) Linux pre-allocate up to 8 blocks on a write to a file. Pre-allocating provides better locality when many writes to independent files are interleaved. T/F

Answer: True

Level:2

5) Which of the following are the attributes of a file?

a) Name
b) Identifier
c) Type
d) Location
e) Size
f) Time, date
g) Protection

1) A,b,d,e
2) A,c,d,e,f
3) all the above
4) all except b

Answer: 3

Level:2

6) FCB stands for _____ (File systems)

Answer: File Control Block

Expl: acronym

Level:1

7) The system uses an extension to indicate the type of the file. The files with _____ are binary executable files.

1) .com
2) .exe
3) .bat
4) Both 1 and 2

Answer: 4

Level: 2

8) Which of the following is a batch file?

1) .com
2) .exe
3) .bat
4) All the above

Answer: 3

Level: 2

9) Which of the following activities connected with file management are OS responsibilities?

1) Creating and deleting files
2) Mapping files onto secondary storage
3) Backing of files on stable storage media
4) All the above

Answer: 4

Exp: by its services

Level:1

10) Information in a file can be accessed by which of the following methods?

a) Sequential Access
b) Direct Access
c) Relative Access
d) None

1) all except a
2) b and c only
3) a, b and c
4) d

Answer: 3

Level:1

11) DFS stands for

1) Dynamic file systems
2) Distributed file systems
3) Direct file systems
4) All the above

Answer: 2

Level: 2

12) WWW stands for_____

Answer: World Wide Web

Level: 2

13) Local file systems can fail because_____

a) Failure of the disk containing the file system
b) Corruption of the directory structure
c) Disk controller failure
d) Cable failure

1) All except a
2) All except d
3) a, b, c and d
4) None of the above

Answer: 3

Level: 2

14) Some systems automatically delete all user files when a user logs off or a job terminates, unless the user explicitly requests that they be kept. What is the advantage of such method?

1) file size is minimized for each user
2) security is provided
3) memory is saved
4) no advantages

Answer: 1,3

When unnecessary files are present this method is useful.

Level: 3

15) Some systems keep all user files unless the user explicitly deletes them. what is the advantage of such a method?

1) file size is minimized for each user
2) security for the user is provided
3) files are not lost
4) no advantages

Answer: 2,3

When the user forgets to save a file this method is useful.

Level:2

16) Some systems allow operations on the type of a file(for instance a database file can be read via an index to a file);while others leave it to the user or simply do not implement multiple file types? Which

system is “better?”

1) systems which keep the track of type of file
2) system which does not implement the type of file
3) it depends on the type of needs of the processes
4) both a and b

Answer: 3

Level:2

17) Some systems support many types of structures for a file’s data, while others simply support a stream of bytes. What are the advantages of systems of first type?

1) Support comes from the system
2) Decreases the size of the system
3) Saves memory space
4) Can implement support presumably more efficiently than an application

Answer: 1,4

Level: 3

18) Some systems support many types of structures for a file’s data, while others simply support a stream of bytes. What are the disadvantages of systems of first type?

1) Increases the size of the system.
2) Applications that may require different file types other than what is provided by the system may not be able to run on such systems.
3) both
4) no disadvantages

Answer: 3

Level:2

19) The purpose of the open() operation is _____

1) used to open a file
2) used to get the status of a file whether it is opened or not
3) it informs the system that the named file is about to become active.
4) none

Answer: 3

Level: 1

20) The purpose of close() operation is _____?

1) used to close a file
2) used to get the status of a file whether a file is closed or not
3) informs the system that the named file is no longer in active use by the user who issued the close operation.
4) none

Answer: 3

Level:1

21) Which of the following is an example of an application in which data in a file should be accessed in a sequential order:

1) Print the content of a record.
2) Print the contents of a file
3) Linear search
4) Binary search

Answer: 2

In a file the contents are accessed in a sequential order.

Level: 2

22) Which of the following is an example of an application in which data in a file should be accessed in a random order:

1) linear search
2) Binary search
3) Print the contents of the file
4) Print the contents of the record

Answer: 4

Records can be accessed in a random fashion.

With the help of record number or equivalent number

Level:1

23) In some systems, a subdirectory can be read and written by an authorized user, just as ordinary files can be. State true or false?_____

Answer: true

Level:1

24) In some systems, a subdirectory can be read and written by an authorized user, just as ordinary files can be. What is the problem that could arise?

1) anybody can modify the file location
2) increases the security of directory
3) the protection scheme gets wasted
4) none

Answer: 1,3

One piece of information kept in a directory entry is file location. If a user could modify this location, then he could access other files defeating the access-protection scheme.

Level: 3

25) In some systems, a subdirectory can be read and written by an authorized user, just as ordinary files can be. Suggest a scheme for dealing with problem faced with this scheme?

1) do not allow users to access the directory/sub-directory.
2) Do not allow the user to directly write onto the subdirectory.
3) allow system operations to write onto sub-directory
4) none

Answer: 2,3

Level:2

26) Consider a system that supports 5000 users. Suppose that you want to allow 4990 of these users to be able to access one file. How would you specify this protection scheme in UNIX?

1) Create an access control list with the names of all 4990 users.

2) Put these 4990 users in one group and set the group access accordingly.
3) create an access control list of 5000 users but 10 users are given no permissions.
4) none

Answer: 1,2

Level: 3

27) Consider a system that supports 5000 users. Suppose that you want to allow 4990 of these users to be able to access one file. Could you suggest protection scheme that can be used more effectively for this purpose than the scheme provided by

UNIX?

1) Create an access control list with the names of all 4990 users.
2) Put these 4990 users in one group and set the group access accordingly.
3) create an access control list of 5000 users but 10 users are given no permissions.
4) none

Answer: 3

Level: 3

28) what are the merits of a scheme of having an access list associated with each file (specifying which users can access the file, and how)

1) memory space is saved
2) easy to modify access control information
3) retrieval time is decreased
4) less overhead when opening a file

Answer: 1,2

Since the access control information is concentrated

At a single place.

Level: 3

29) what are the merits of having a scheme *user control list* associated with each user (specifying which files a user can access, and how).

1) memory space is saved

2) easy to modify access control information
3) retrieval time is decreased
4) less overhead when opening a file

Answer: 4

Level:2

30) Consider a system that supports 8535 users. Suppose that you want to allow 6999 of these users to be able to access one file. How would you specify this protection scheme in UNIX?

1) create a user control list(specifying which files a user can access and how) of respective file names.
2) create a file control list(specifying which files a user can access and how) with all 6999 user names.
3) create a group of 6999 users and set permissions accordingly to the group.
4) create a group of 8535 users with 1536 users having no permissions.

Answer: 2,3

Level: 3

31) Consider a system that supports 8535 users. Suppose that you want to allow 6999 of these users to be able to access one file. Could you suggest protection scheme that can be used more effectively for this purpose than the scheme provided by

UNIX?

1) create a user control list(specifying which files a user can access and how) of respective file names.
2) create a file control list(specifying which files a user can access and how) with all 6999 user names.
3) create a group of 6999 users and set permissions accordingly to the group.
4) create a group of 8535 users with 1536 users having no permissions.

Answer: 4

Level: 3

32) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for contiguous, linked, and indexed (single-level) allocation strategies, if, the block is added at the

beginning. In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 100,1,201
2) 200,100,1
3) 101,201,1
4) 201,1,1

Answer: 4

Level: 3

33) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for linked,contiguous,and indexed (single-level) allocation strategies, if, the block is added at the middle. In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 101,50,1
2) 51,1,101
3) 101,52,1
4) 1,101,52

Answer: 3

Level: 3

34) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed (single-level) and linked allocation strategies, if, the block is added at the middle.

1) 101,52
2) 1,52
3) 52,1
4) 1,101

Answer: 2

Level: 2

35) Consider a file currently consisting of 100 blocks. Assume that the file control block

(and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for linked, contiguous, and indexed (single-level) allocation strategies, if, the block is added at the end. In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 101,52,1
2) 1,52,1
3) 1,3,1
4) 3,1,1

Answer: 4

Level: 3

36) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for contiguous, linked, and indexed (single-level) allocation strategies, if, the block is removed from the end.In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 0,0,101
2) 0,100,0
3) 0,0,100
4) 100,0,0

Answer: 2

Level: 2

37) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for contiguous, linked, and indexed (single-level) allocation strategies, if, the block is removed from the beginning. In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 198,1,0
2) 201,101,1
3) 101,0,198
4) 198,0,0

Answer: 1

Level: 2

38) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed, linked and contiguous allocation strategies, if, the block is removed from the middle. In the contiguous-allocation case, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 98,52,0
2) 0,52,101
3) 0,52,98
4) 52,0,98

Answer: 3

Level: 2

39) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for contiguous allocation

if, the block is added at the middle, at the end, at the beginning. Assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 198,1,101
2) 201,1,101
3) 1,101,201
4) 101,1,201

Answer: 4

Level: 2

40) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed (single-level) allocation

if, the block is removed from the beginning

from the end,from the middle

1) 1,1,1
2) 1,0,1
3) 0,0,0

4) 101,0,52

Answer: 3

Level: 2

41) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for linked allocation

if, the block is added at the end.

1) 52
2) 101
3) 3
4) 1

Answer: 3

Level: 2

42) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for linked allocation

if, the block is added in the middle and

block is removed from the middle

1) 52,52
2) 53,52
3) 52,52
4) 1,52

Answer: 3

Level: 2

43) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for contiguous allocation

if, the block is removed from the beginning and

from the end. Assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 198,98
2) 98,101

3) 52,98
4) 98,52

Answer: 1

Level: 3

44) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for linked allocation

if, the block is added at the beginning

if the block is removed from the beginning and from the end

1) 1,1,100
2) 1,0,101
3) 1,0,100
4) 0,0,100

Answer: 1

Level: 2

45) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed (single-level) allocation if, the block is added at the beginning, middle and end.

1) 0,0,0
2) 1,1,1
3) 1,0,1
4) 0,0,1

Answer: 2

Level: 2

46) Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed (single-level) allocation

if, the block is added at the beginning, removed from the end

1) 0,1
2) 1,0

3) 0,0
4) 1,1

Answer: 2

Level: 2

47) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for linked allocation

if, the block is added and removed at/from the beginning

1) 0,0
2) 1,1
3) 1,0
4) 0,1

Answer: 2

Level: 2

48) Consider a file currently consisting of 500 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for contiguous allocation if the block is added at the middle. Assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 501
2) 251
3) 499
4) 249

Answer: 1

Level: 1

49) Consider a file currently consisting of 250 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for linked allocation if, the block is removed from the end.

1) 249
2) 250
3) 501
4) 0

Answer: 2

Level: 2

50) Consider a file currently consisting of 300 blocks. Assume that the file control block (and the index block, in the case of indexed allocation)

is already in memory. Calculate how many disk I/O operations are required for indexed (single-level) allocation if, the block is added at the end and removed from middle.

1) 300,0
2) 301,1
3) 1,0
4) 0,1

Answer: 3

Level: 2

51) Consider a file currently consisting of 100 blocks. Assume that the file control block is already in memory. Calculate how many disk I/O operations are required for contiguous allocation if the block is added at the end, assume that there is no room to grow in the beginning, but there is room to grow in the end.

1) 0
2) 1
3) 52
4) 101

Answer: 2

Level: 2

52**)** What problems could occur if a system allowed a file system to be mounted simultaneously at more than one location?

1) increases the retrieval time for the files.
2) decreases the retrieval time for the files
3) there would be multiple paths to the same file
4) nothing happens

Answer: 3

It confuses the users or encourage mistakes (deleting a file with one path deletes the file in all the other paths).

Level:2

53) Can we mount more than one file system at different places in a system at a time. Yes/no

Answer: yes

Level:2

54) What happens if we mount more than one file system at a single place in a system.

1) file system gets overwrite
2) the two file systems gets installed at different places but only one system works always.
3) the two file system gets installed at different places with two working one after another
4) we cannot install like that.

Answer: 1

Level: 2

55) Why must the bitmap for file allocation be kept on mass storage, rather than in main memory?

1) to keep the free space list not to lost in case of system crash
2) to keep a backup of bitmap of the file
3) both
4) none

Answer: 1

Level:2

56) If a file is usually accessed sequentially and is large then which of the following file system is preferable?

1) indexed
2) linked
3) contiguous
4) none

Answer: 2

As in the linked system files are accessed by a link.

Level:1

57) If a file is usually accessed sequentially and is relatively small, then which of the following file system is preferable?

1) contiguous
2) linked
3) indexed
4) none

Answer: 1

Level:1

58) If a file is usually accessed randomly and is large ,then which of the following file system is preferable?

1) contiguous
2) linked
3) indexed
4) none

Answer: 3

Level:1

59) what is the problem with contiguous allocation compared to standard contiguous allocation

1) user must pre-allocate enough space for each file
2) the file size cannot be increased eventhough it is overflowed
3) both a and b
4) none

Answer: 3

Level:2

60) what is the advantage of standard contiguous allocation over common contiguous allocation?

1) less overhead
2) more overhead
3) file size is variable
4) none

Answer: 1

Level: 2

61) Caches help improve system performance?
True/false

Answer: true

Cache store information in a temporary storage for faster retrieval.

Level:1

62) How caches help improve system performance?

1) allows components of differing speeds to communicate more efficiently
2) by storing data from the slower device, temporarily,in a faster device (the cache).
3) Both 1and 2
4) None

Answer: 3

Level: 2

63) Instead of more uses of caches why they are not used much?

1) more caches decreases performance
2) caches are expensive than the device they are caching for
3) cache memory has a limited size
4) both 2 and 3

Answer: 4

Level:2

64) What are the advantages of dynamic tables?

1) allows more flexibility in system use growth
2) tables are never exceeded, avoiding artificial use limits
3) Both 1 and 2
4) None

Answer: 3

Level:2

65) What are the penalties to the operating system for dynamically allocating tables?

1) kernel structures and code are more complicated
2) more potential for bugs
3) the use of one resource can take away more system resources

4) all the above

Answer: 4

Level:2

66) Dynamic allocations of tables are better than static allocation. state true or false.

Answer: true

More flexibility.does not allow files to exceed limits.

Level:2

67) Which of the following are the limitations of 32bit FAT file system?

1) Fragmentation
2) Size limitation of 2 GB
3) Lack of access protection for files
4) All

Answer: 4

Level: 2

68)The following features belong to which file system?

Data recovery

Security

Fault tolerance

Large files and file system

Multiple data streams

Sparse files

Encryption

1) 16 bit FAT
2) 32 bit FAT
3) NTFS
4) All

Answer: 3

Level: 3

69) FTP stands for____

1) Fixed transfer protocol
2) File transfer protocol
3) Fun transfer protocol
4) None

Answer: 2

Level:1

70) Which of the fields about a file are maintained by OS

1) Identifier

2) Type

3) Location

4) all the above

Answer:4

Exp: these file details are stored in OS

Level:1

71) Which of the following are the basic file operations?

a) creating a file

b) writing into a file

c) Reading a file

d)Repositioning within a file

e) Deleting a file

f) Truncating a file

1) all except d

2) all except d and f

3) all

4) none

Answer: 3

Exp: operations of a file

Level: 1

72) .mpeg is of type _____ file

1) text

2) batch

3) multimedia

4) library

Answer: 3

Exp: as per its service

Level: 2

73) “autoexec” file has an extension _____

1) .exe

2) .sh

3).bat

4) .mp3

Answer: 3

Exp: as per its extension

Level:2

74) Which file access type is also known as relative access?

1) Sequential access

2) Direct access

3) Both

4) None

Answer: 2

Exp: by def

Level: 1

75) Which of the following are types of directories?

1) single level directories.

2) tree structured directories.

3)Acyclic graph directories

4) all the above

Answer:4

Exp: list of directories

Level:1

76) Which of the following are types of directories?

1) single level directories.

2 two level directories.

3) Acyclic graph directories

4) all the above

Answer:4

Exp: list of directories

Level:1

77) Path name beginning at the root and follows a path down to the specific file, giving the directory names on the path

1) absolute path name

2) relative path name

3) both

4) None

Answer: 1.

Exp: by def

Level: 2

78) Which of the following are types of directories?

1) single level directories.

2 General Graph Directory

3) Acyclic graph directories

4) all the above

Answer:4

Exp: list of directories

Level: 1

79) Which of the following are the File allocation methods?

1) Contiguous Allocation

2)Linked Allocation

3) Indexed allocation

4) All the above

Answer: 4

Exp: list of file allocation

Level:1

80) VFS stands for _____ (File Systems)

Answer: Virtual File Systems

Exp: acronym

Level: 1

81) FAT stands for _____ (File systems)

Answer: File Allocation Table

Exp: acronym

Level:1

82) File is a collection of _____

1) Sectors

2) Blocks

3) Records

4) Tracks

Answer: 3

level: 1

83) Which of the following file name extension suggests that the file is Backup copy of another file?

1) TXT
2) COM
3) BAS
4) BAK

Ans: 4

level: 2

84) A tree sturctured file directory system

1) allows easy storage and retrieval of file names
2) is a much debated unnecessary feature
3) is not essential when we have millions of files
4) none of the above

Ans: - 1

level: 1

85) _____ begins at the root and follows a path down to the specified file

1) Relative path name
2) Absolute path name
3) Standalone name
4) All of the above

Ans: - 2

level: 2